

flight_recorder — Design and User Guide

Lean, crash-tolerant flight-data logging for ROS 2 control nodes: preallocated column buffers, one HDF5 format, identical Python and C++ writers

evannsmc

2026-09-28

Table of contents

1	Introduction	3
2	Why a dedicated flight logger?	4
3	Architecture and data model	5
3.1	The pieces	5
3.2	Four kinds of data	6
4	The file format (version 1)	6
4.1	Layout	6
4.2	Streams	7
4.3	Records	8
4.4	Events	9
4.5	Root metadata	9
4.6	Reading without this library	9
5	Memory model: why append is cheap	9
5.1	Python: ColumnBuffer	9
5.2	C++: fr::Stream	11
6	Threading	11
7	Incremental flushing and crash tolerance	12
7.1	What a flush does	12
7.2	Autosave (Python)	12
7.3	Caveats	13
8	Python API	13
8.1	Recorder	13
8.2	ColumnBuffer	14
8.3	FlightLog	14
8.4	A complete example	15
8.5	Analysis recipes	15

9 C++ API	17
10 ROS 2 integration	18
11 Command line and CSV export	19
12 ROS2Logger compatibility and migration	19
13 Benchmarks	20
14 Testing	22
15 Limitations, gotchas and FAQ	23
15.1 By design	23
15.2 Behaviours to know about	23
15.3 FAQ	24
16 Reproducing this guide	24

1 Introduction

`flight_recorder` is a small library for logging what a robot's control node *computes*, from inside the node, at control rate, without disturbing the control loop. It has two writers (Python and header-only C++) that produce the **same HDF5 file layout**, and a Python reader that loads that layout into pandas.

This guide explains the whole package. It covers why it exists, the on-disk format, the memory and threading model behind the fast append, how incremental flushing makes it crash-tolerant, every public Python and C++ API, the ROS 2 patterns, the command line, the ROS2Logger compatibility layer, the benchmarks, the tests, and the limitations. Everything below matches version **0.1.2** of the code and **format version 1**. All figures were generated by `docs/guide/make_figures.py` from the real library.

The whole API in three snippets (from the README):

```
from flight_recorder import Recorder

rec = Recorder('flight.h5', metadata={'robot': 'skie_2'}, autosave_period=5.0)
ticks = rec.stream('ticks', ['time', 'x', 'y', 'z', 'yaw', 'thrust']) # once, at init

ticks.append(t, x, y, z, yaw, thrust) # every tick
rec.record('plans', seq, {'tube': tube, 'ref': ref}, {'t_start': t}) # big arrays:
    ↳ only when they change
rec.event(t, 'backup', 'no certified plan') # rare,
    ↳ timestamped strings

rec.save() # at shutdown
    ↳ (final flush)

#include <flight_recorder/recorder.hpp>

fr::Recorder rec("flight.h5", {"robot", std::string("skie_2")});
auto& ticks = rec.stream("ticks", {"time", "x", "y", "z", "yaw", "thrust"});
ticks.append(t, x, y, z, yaw, thrust); // ~15 ns
rec.record("plans", seq, {"tube", fr::Array(tube.data(), {rows, 10})}, {"t_start",
    ↳ t});
rec.save();

from flight_recorder import FlightLog

with FlightLog('flight.h5') as log:
    df = log['ticks'] # pandas DataFrame
    tube = log.record('plans', 12)['tube'] # a stored array
    print(log.summary())
```

How to read this guide. If you only want to use the library, read sections 3, 8, 10 and 15. For the file format read section 4. For why it is fast and safe, read sections 5 and 7. The shorter repository documents are complementary: [docs/DESIGN.md](#) (design notes), [docs/COMPARISON.md](#) (against other logging methods) and [docs/MIGRATION.md](#) (moving from ROS2Logger).

2 Why a dedicated flight logger?

Research control nodes usually log by appending to Python lists: ROS2Logger's LogTypes, a list of rows, or a list of dicts. They then write a CSV at shutdown. In a 100 Hz controller that has three costs that are easy to miss.

1. Garbage-collector stalls. CPython frees most objects by reference counting. Container objects (lists, dicts, tuples, class instances) are *also* tracked by a cyclic garbage collector that periodically scans them looking for reference cycles. The collector is generational. A *full* collection scans every tracked object in the process and holds the GIL while it runs, so every Python thread freezes, including the control loop. Full collections become both slower and more frequent as long-lived tracked objects accumulate: CPython triggers one when that population has grown by 25 %. A logger that keeps one list per tick feeds exactly this. In the RTA-MM-GPR node that motivated the library, logging lists were about 97 % of the tracked objects at the end of a flight. Full collections took **≈300 ms each, 4–5 times per flight**, which is 30 missed control ticks each time.

2. Memory. A Python float is a 24-byte object, plus an 8-byte pointer in the list that holds it. A list of floats therefore costs about 4× the data, and nested lists of rows more. In the benchmark (section 13), ROS2Logger holds 42 MB for 4.8 MB of numbers.

3. Nothing reaches the disk until shutdown. A crash, `kill -9`, a failed `destroy_node()` or a power loss loses the whole flight.

`flight_recorder` removes all three:

- each stream lives in **one preallocated float64 array**. `append` copies the numbers into it and keeps no Python objects, so the collector never sees the data (Figure 1);
- large arrays that change rarely (plans, trajectories, gain matrices) are **stored once, when they change**, instead of being re-logged every tick;
- everything goes to **one self-describing HDF5 file** that is **flushed incrementally**. With `autosave_period=5.0`, a crash loses at most 5 s.

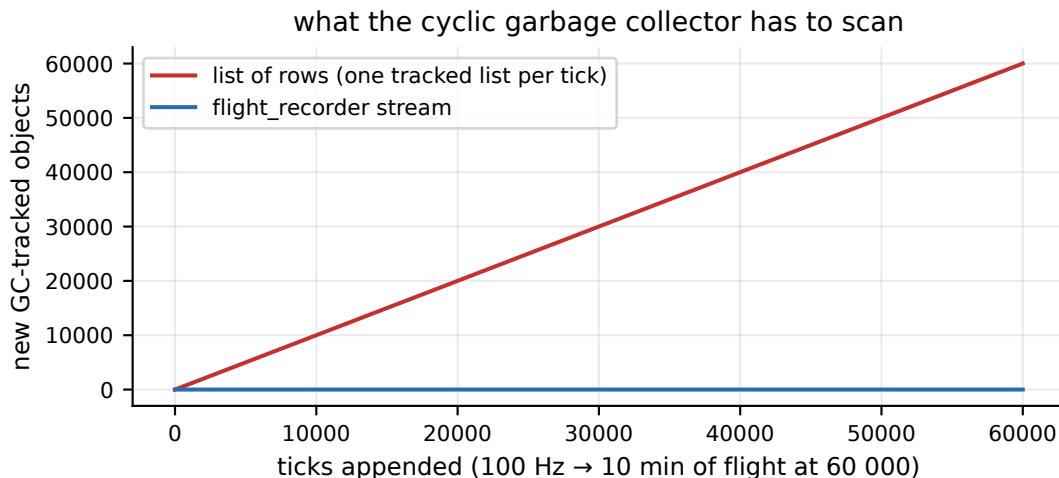


Figure 1: New objects the cyclic GC must track while logging 60 000 ticks of 4 values. The list-based logger adds one tracked list per tick. The `flight_recorder` stream, which also grows by doubling from 1024 rows, adds a handful (4 in this run). Measured with `gc.get_objects()` while the GC was disabled.

3 Architecture and data model

3.1 The pieces

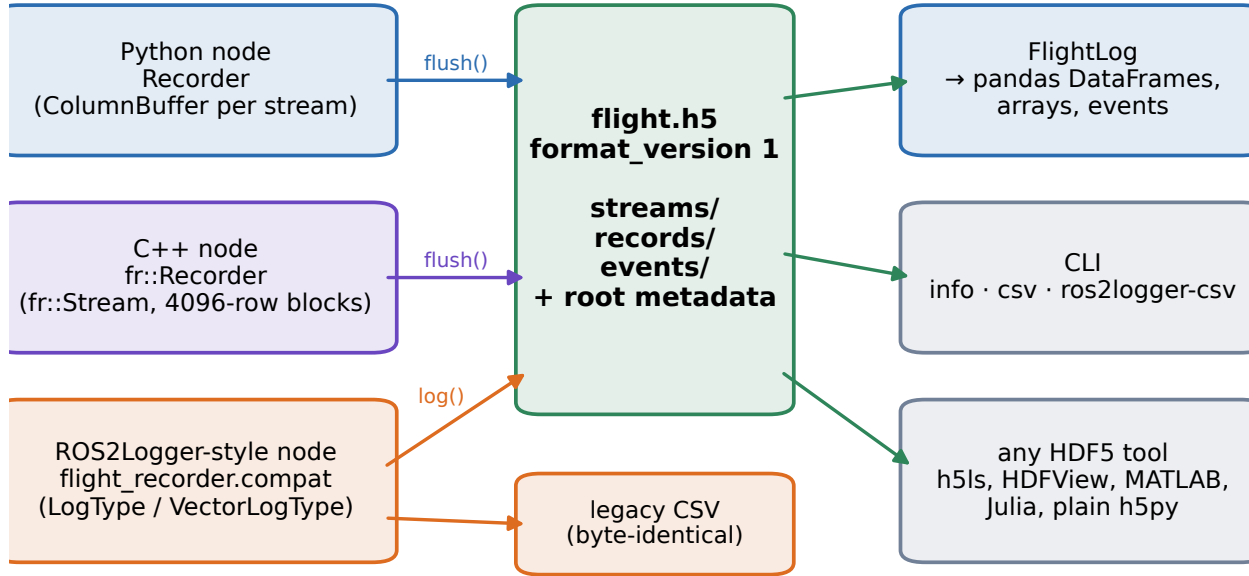


Figure 2: Package architecture. Three kinds of writer produce one file format. The Python reader, the CLI and any HDF5 tool read it.

Table 1: Repository map.

file	role
flight_recorder/format.py	the normative constants: names of the three groups, <code>FORMAT_VERSION = 1</code> , <code>CHUNK_ROWS = 4096</code> , <code>GZIP_LEVEL = 4</code>
flight_recorder/buffer.py	ColumnBuffer: the append-only float64 table behind every Python stream
flight_recorder/recorder.py	Recorder (collect + flush), record_key, git_commit
flight_recorder/reader.py	FlightLog: read either writer's files into DataFrames and arrays
flight_recorder/csv_export.py	plain CSV and ROS2Logger-layout CSV export
flight_recorder/compat.py	ROS2Logger-compatible Logger, LogType, VectorLogType, install_shutdown_logging
flight_recorder/__main__.py, scripts/flight_recorder	command line (info, csv, ros2logger-csv)
include/flight_recorder/recorder.hpp	header-only C++ writer (<code>fr::Recorder</code> , <code>fr::Stream</code> , <code>fr::Array</code>) on the HDF5 C API
examples/, benchmarks/, test/	a ROS 2 node, a C++ program, the benchmark harness, 17 tests

The package builds with colcon as an `ament_cmake` package. It installs the Python module, the

flight_recorder script, the header, and the CMake target `flight_recorder::flight_recorder`. Outside ROS, `pip install -e .` installs the Python part (`pyproject.toml`; dependencies `numpy`, `h5py`, `pandas`).

3.2 Four kinds of data

A flight produces data of different shapes at different rates, and each is stored the way it is produced:

Table 2: The data model.

kind	example	API	stored as
stream	time, state, commands, timings: fixed columns, one row per tick	<code>rec.stream(name, columns), then .append(...)</code>	one float64 dataset per column
record	a plan, a trajectory, a gain matrix: arrays that change rarely	<code>rec.record(group, key, arrays, attrs)</code>	one HDF5 group per record
event	“backup engaged”, a mode switch: rare, timestamped strings	<code>rec.event(t, kind, detail)</code>	three parallel datasets
metadata	robot, parameters, git commit, host, date	<code>Recorder(metadata=...), set_metadata(...)</code>	root attributes

The stream/record split is the main design idea. The RTA node that motivated the library logged a 12×4 slice of its reachable tube *on every control tick*, because a per-tick table was the only shape its logger had. The tube only changed when a new plan arrived, about 1 tick in 30. With records, each plan is stored **once, in full**, and each tick stores only the plan number (and row) it used. On a real flight that was 5.42 MB $\rightarrow$ 1.61 MB, while also keeping more information: the full tube instead of a slice.

Streams are declared once with fixed column names. Use one stream per *rate or producer* (control ticks at 100 Hz, an estimator at 10 Hz, ...) instead of one wide table. Align them afterwards with `pandas.merge_asof` (section 8.5).

4 The file format (version 1)

4.1 Layout

```
flight.h5
├─ attrs: format="flight_recorder", format_version=1, writer="python"|"cpp",
│      created (ISO 8601, local time, seconds), host, <user metadata>
├─ streams/
│   └─ <stream>/                attrs: columns = [names, in order], n_rows
│       └─ <column>              float64 (n_rows,), chunks of 4096 rows, shuffle + gzip
│   ↳ 4, resizable
├─ records/
│   └─ <group>/
│       └─ <key>/                attrs: the attrs given to record(...)
│           └─ <name>              float64 array of any shape (shuffle + gzip above 256
│   ↳ elements)
└─ events/
```

```

└─ time                float64 (n,)          chunks of 256, resizable,
↪ uncompressed
└─ kind                UTF-8 string (n,)      chunks of 256, resizable,
↪ uncompressed
└─ detail              UTF-8 string (n,)      chunks of 256, resizable,
↪ uncompressed

```

This is the layout `flight_recorder/format.py` documents, and both writers produce it. Below is the actual layout of the synthetic 60 s flight that `make_figures.py` records with the Python writer. It is walked with `h5py`, and only 2 of the 40 plan records are shown:

```

/   @ controller='twinflight', created='2026-09-28T17:23:30', format='flight_recorder'
   @ format_version=1
   @ gains=[6.0, 4.5, 1.0]
   @ host='example-host'
   @ rate_hz=100.0
   @ robot='skie_2'
   @ writer='python'
events/
  detail    str      (3,)          chunks=(256,) resizable
  kind      str      (3,)          chunks=(256,) resizable
  time      float64 (3,)          chunks=(256,) resizable
records/
  plans/
    000000/  @ horizon_s=3.0, t_start=0.0
              reference float64 (60, 4)
    000001/  @ horizon_s=3.0, t_start=1.5
              reference float64 (60, 4)
    ... (38 more)
  streams/
    estimator/ @ columns=['time', 'bias_x', 'bias_y'], n_rows=600
      bias_x    float64 (600,)      chunks=(4096,) gzip4+shuffle resizable
      bias_y    float64 (600,)      chunks=(4096,) gzip4+shuffle resizable
      time      float64 (600,)      chunks=(4096,) gzip4+shuffle resizable
    ticks/     @ columns=['time', 'x', 'y', 'z', 'x_ref', 'y_ref', 'z...'],
↪ n_rows=6000
      plan_seq  float64 (6000,)      chunks=(4096,) gzip4+shuffle resizable
      thrust    float64 (6000,)      chunks=(4096,) gzip4+shuffle resizable
      time      float64 (6000,)      chunks=(4096,) gzip4+shuffle resizable
      x         float64 (6000,)      chunks=(4096,) gzip4+shuffle resizable
      x_ref     float64 (6000,)      chunks=(4096,) gzip4+shuffle resizable
      y         float64 (6000,)      chunks=(4096,) gzip4+shuffle resizable
      y_ref     float64 (6000,)      chunks=(4096,) gzip4+shuffle resizable
      z         float64 (6000,)      chunks=(4096,) gzip4+shuffle resizable
      z_ref     float64 (6000,)      chunks=(4096,) gzip4+shuffle resizable

```

4.2 Streams

- **Column-major on disk.** Each column is its own 1-D dataset. Reading one signal (`log.column('ticks', 'z')`) touches only that dataset, and building a `DataFrame` needs no reshuffling.

- **Order and length live in attributes.** The group attribute columns holds the names in declaration order. HDF5 lists members alphabetically, so the reader always uses this attribute. `n_rows` is the number of rows written so far, updated on every flush.
- **Chunked, resizable, compressed.** Every column is created with `maxshape=(None,)`, `chunks=(4096,)` (32 KiB of float64 per chunk), the **shuffle** filter and **gzip**. The level is `compression_level`, 4 by default. The shuffle filter regroups the bytes of consecutive values: all first bytes together, then all second bytes, and so on. For smooth signals the high-order bytes (sign, exponent, leading mantissa) repeat, and gzip then compresses them well.
- **Everything is float64.** Integers and booleans are exact up to 2^{53} . Missing values are NaN.

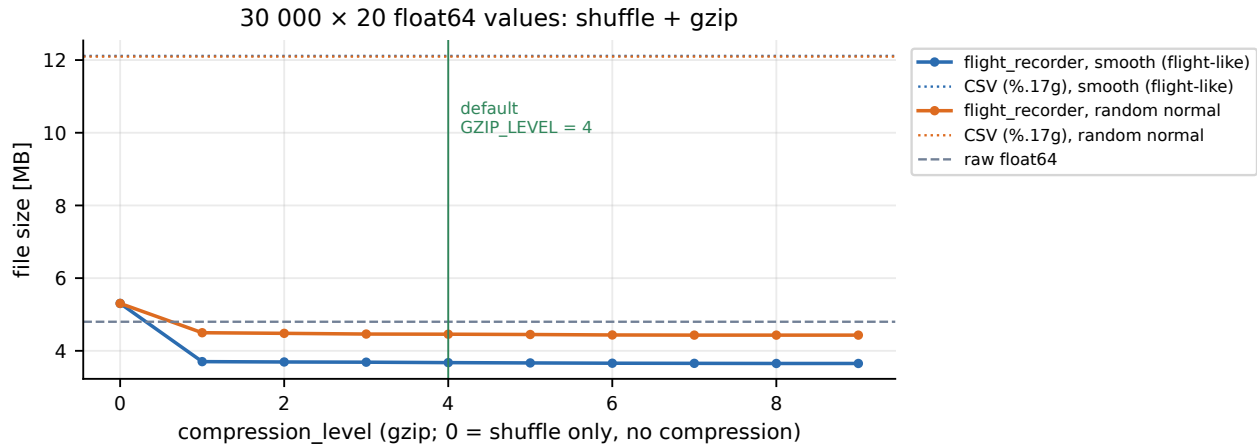


Figure 3: File size of one $30\,000 \times 20$ stream against the gzip level, for flight-like smooth signals (with a small noise floor) and for random normal data, compared with raw float64 and with CSV written at full precision. Level 0 is *larger* than raw: with nothing compressed, the last, partly filled 4096-row chunk of each column is stored whole (32 768 rows of storage for 30 000). Most of the gain comes at level 1, and the default of 4 is on the plateau. On real flight data the effect is larger: 0.36 MB against 1.28 MB of CSV for a 2000×34 table (README).

4.3 Records

- Stored under `records/<group>/<key>/`, one group per record, one dataset per array. Arrays can have any shape (converted to float64).
- **Integer keys are zero-padded to six digits** (`record_key(12) == '000012'`; C++ uses `%06lld`). HDF5 iterates alphabetically, so the padding makes that order numeric. String keys are used as they are.
- Arrays with **more than 256 elements** get shuffle + gzip. Smaller ones are stored contiguously, where the filter overhead is not worth it. (Python lets h5py pick the chunk shape; C++ uses one chunk of the array's full shape.)
- **Re-recording a key replaces it.** The flush deletes the old group before writing the new one (`test_rerecording_a_key_replaces_it`).
- Record attributes accept scalars, strings and small arrays. The conversion rules are in 1.

Table 3: How Python metadata and record attributes are written (`recorder._attr_value`). C++ attribute values are a `std::variant<double, int64_t, std::string, std::vector<double>>`.

Python value	stored as
<code>bool</code> / <code>numpy.bool_</code>	integer 0/1
<code>str</code> , <code>int</code> , <code>float</code> , <code>NumPy scalar</code>	as is
<code>list</code> / <code>tuple</code> / <code>ndarray</code> of numbers or booleans	float64 array
<code>list</code> / <code>tuple</code> / <code>ndarray</code> of anything else	array of UTF-8 strings
<code>None</code>	skipped (not written)
anything else	<code>str(value)</code>

4.4 Events

Three parallel 1-D datasets. `time` is float64, and `kind` and `detail` are variable-length UTF-8 strings. All three are resizable with chunks of 256 and uncompressed, and a flush appends to all three together. The events group and its datasets exist even if no event was ever logged (length 0).

4.5 Root metadata

The writer always sets `format`, `format_version`, `writer` ("python" or "cpp"), `created` (local time, ISO 8601 to the second) and `host`. User metadata is merged on top. In Python, `Recorder(metadata=...)` is applied *after* the defaults, so it can override them, e.g. `host`. Metadata is rewritten on every flush, so `set_metadata` late in the flight still reaches the file.

4.6 Reading without this library

The file is plain HDF5. It can be read with `h5ls -r flight.h5`, `h5dump`, `HDFView`, `MATLAB h5read`, `Julia HDF5.jl`, or `h5py` alone:

```
import h5py, numpy as np
with h5py.File('flight.h5', 'r') as f:
    g = f['streams/ticks']
    cols = [c.decode() if isinstance(c, bytes) else c for c in g.attrs['columns']]
    data = {c: g[c][()] for c in cols} # dict of float64 arrays
    plan = f['records/plans/000012/tube'][()]
```

Compatibility. FlightLog accepts files whose root attribute `format` equals "flight_recorder" and whose `format_version` is at most the one it supports (1). A file from a newer writer raises a `ValueError` that names both versions, instead of being misread. `test_cpp_roundtrip.py` checks that the C++ writer's output reads back bit-exactly with the Python reader.

5 Memory model: why append is cheap

5.1 Python: ColumnBuffer

A `ColumnBuffer` is (`capacity × columns`) float64, initialised to NaN, plus a row counter `_n`:

```
def append(self, *values):
    if self._n == self._data.shape[0]:
```

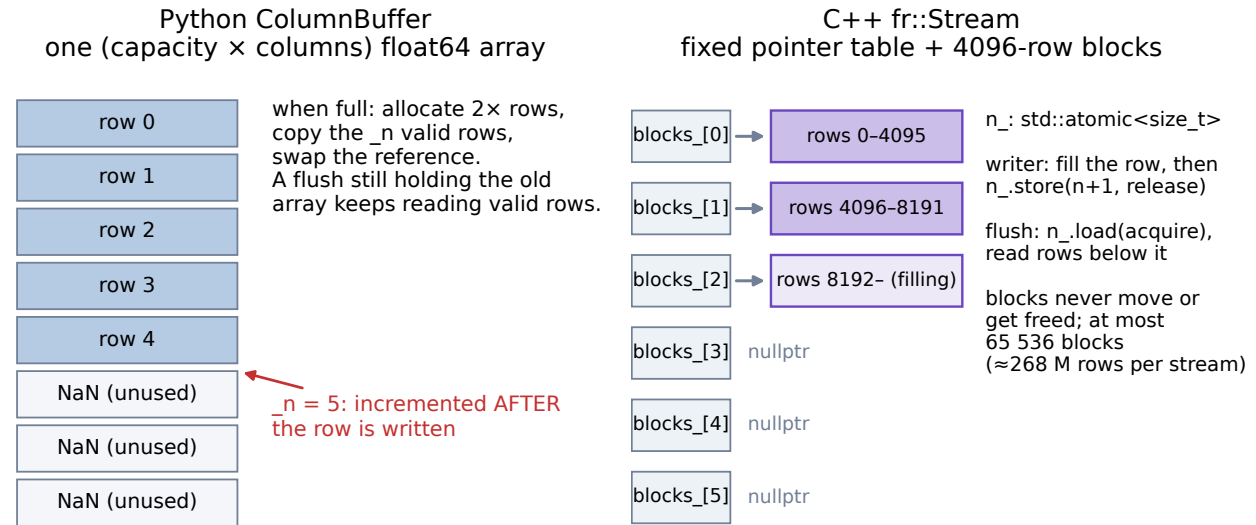


Figure 4: The two stream buffers. Left: Python keeps one NumPy array per stream and doubles it when full. Right: C++ keeps a fixed table of pointers to 4096-row blocks that never move.

```

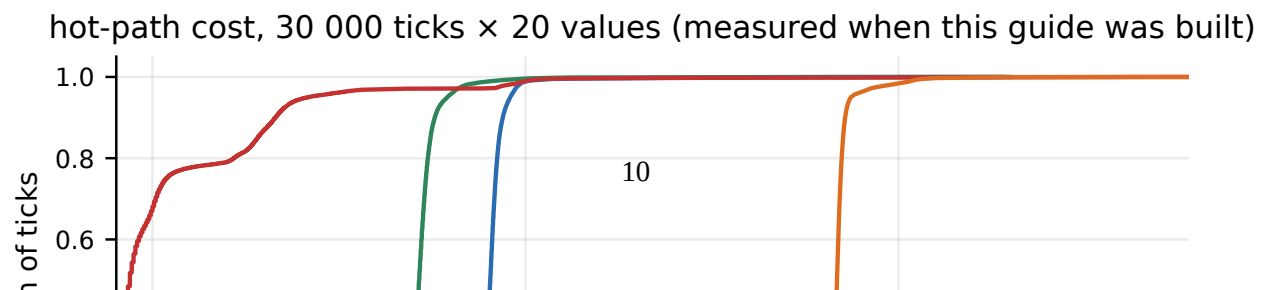
self._grow()                # new array of 2x rows, copy the _n valid rows,
    ↳ swap
self._data[self._n] = values # the numbers are copied into the array
self._n += 1                # published AFTER the row is written

```

- **No per-row objects survive.** The argument tuple and the float objects are freed by reference counting as soon as `append` returns. Nothing is added to what the cyclic GC tracks; `test_gc.py` asserts fewer than 100 new tracked objects over 100 000 appends (Figure 1).
- **Growth by doubling** (amortised $O(1)$). Pass `capacity ≈` the flight length to never grow in flight: $100 \text{ Hz} \times 600 \text{ s} = 60\,000$ rows, which is 480 KB per column. `Recorder.stream` defaults to 4096 rows; a bare `ColumnBuffer` defaults to 1024.
- **Two hot-path forms.** `append(*values)` unpacks arguments. `append_array(row)` takes a sequence or array that is already built, and is about twice as fast for wide rows (Table 4).
- The row width is checked by NumPy: a wrong number of values raises `ValueError(test_rejects_wrong_row_width)`.
- Readers: `column(name)` returns a **view** of the valid rows; `rows(start, stop)`, `as_dict()` and `to_dataframe()` return copies.

Table 4: Hot-path cost measured by `make_figures.py` when this guide was built.

hot path (20 values)	p50	p99
<code>Stream.append(*values)</code>	0.80 μs	1.01 μs
<code>Stream.append_array(row)</code>	0.52 μs	0.82 μs
<code>list.append(list of 20)</code>	0.09 μs	1.00 μs
compat: $20 \times \text{LogType.append}$	6.84 μs	10.79 μs



Absolute numbers depend on the machine and its load. The benchmark in section 13 measures `append_array` at 0.52 μ s p50 / 1.6 μ s p99 in a clean, isolated process.

5.2 C++: `fr::Stream`

```
template <typename... Ts> void append(Ts... values); // static_cast<double> each,
↳ then append_row
void append_row(const double* row, size_t count) {
    const size_t n = n_.load(std::memory_order_relaxed);
    double* dst = ensure_block(n / kBlockRows) + (n % kBlockRows) * width_;
    for (size_t i = 0; i < width_; ++i) dst[i] = row[i];
    n_.store(n + 1, std::memory_order_release); // publish the complete row
}
```

- Rows live in **blocks of `kBlockRows = 4096` rows** (row-major). A fixed table of `kMaxBlocks = 65 536` block pointers is allocated when the stream is created, so a stream holds at most ≈ 268 M rows (`std::length_error` beyond that). The table itself is 512 KiB of pointers.
- Blocks are **never moved or freed** while the recorder exists, unlike a `std::vector`, which reallocates as it grows. That is what makes a concurrent flush safe (section 6).
- The constructor preallocates enough blocks for capacity rows (default 4096). Beyond that one block is allocated every 4096 rows, the only allocation on the hot path.
- A wrong number of values throws `std::invalid_argument`.
- Cost: **≈ 15 ns per append** (cpp_bench: 0.016 μ s p50 and 2.2 μ s p99 when rebuilt for this guide; the p99 is the one tick in 50 that also copies a 100×10 plan into record).

6 Threading

The rules are the same in both languages:

- **One writer thread per stream**, normally the control loop. Different streams may be written by different threads.
- **`flush()` may run on any other thread at the same time**. Only one flush runs at a time; a second one waits for the flush lock.
- `record()`, `event()` (and, in C++, `set_metadata()`) take a short mutex. They are meant for rare data, not for every tick.

Why a concurrent flush only reads complete rows:

- **Python.** `_n` is incremented after the row is written, and under the GIL the flusher sees those two steps in that order. `rows(start)` reads `_n` once and copies the rows below it. If the writer grows the buffer during a flush, the flusher still holds the old array, whose first `_n` rows are complete.
- **C++.** The writer stores `n_` with `memory_order_release` after writing the row. The flusher loads it with `memory_order_acquire(size())`, so every row below it is visible. Blocks never move, so a pointer to an old block stays valid.

Both are tested under stress. In Python, 50 000 appends run while another thread flushes in a tight loop, and every value is checked. In C++, `cpp_example --concurrent-flush` flushes every 2 ms while writing 20 000 rows, and `test_cpp_roundtrip.py` checks every value.

7 Incremental flushing and crash tolerance

7.1 What a flush does

```
flush():
    take the flush lock
    snapshot (under the data lock): the stream list, records and events not yet flushed
    open the file: mode "w" on the first flush (replaces any old file), "a" afterwards
    write all metadata attributes
    for each stream: new rows = rows [flushed_rows, n) -> resize every column dataset,
    ↪ write the new rows,
        flushed_rows = n, attrs n_rows = n
    write the new records (replacing existing keys) and append the new events
    close the file
```

Every flush **closes the file**, and `save()` is simply the last flush. Two consequences:

- **Between flushes the file is closed and consistent.** It can be opened with FlightLog while the node is still flying, e.g. for a live plot every few seconds (`test_autosave_writes_periodically` does exactly this).
- **A crash loses only what was not flushed yet.** `test_crash_after_flush_keeps_flushed_data` runs a process that flushes 1000 rows, appends 1000 more and dies with `os._exit(1)` (no `save()`, no `atexit`, no `finally`). The file contains exactly the first 1000.

`test_incremental_flush_equals_single_save` checks that many small flushes produce the same data as one save.

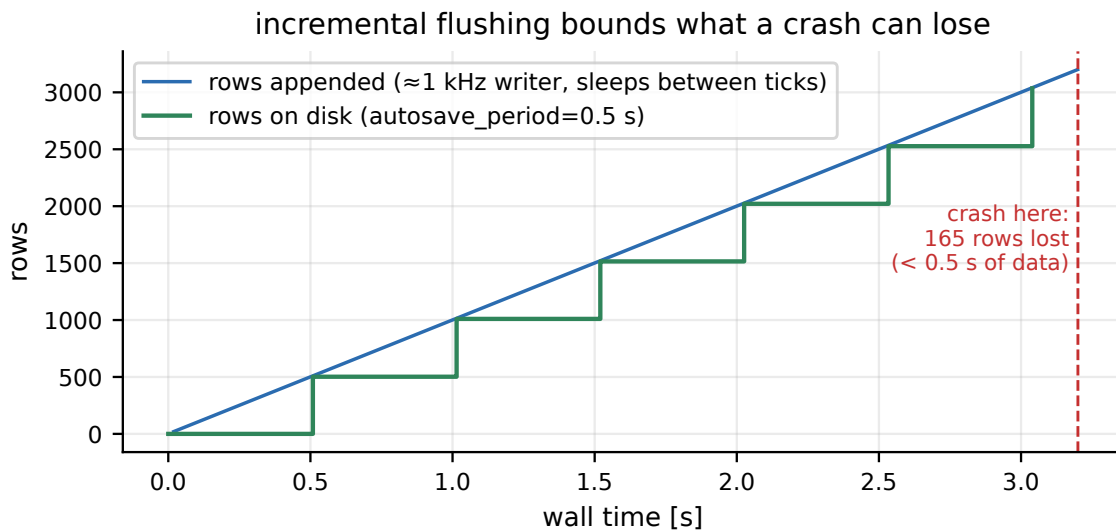


Figure 6: Rows appended by a writer at about 1 kHz (sleeping between ticks, like an rclpy timer) against rows safely on disk with `autosave_period=0.5`. The flushes happen on the recorder's background thread. If the process died at the red line, everything below the green staircase would survive.

7.2 Autosave (Python)

`Recorder(path, autosave_period=5.0)` starts a daemon thread named `flight_recorder_autosave` that calls `flush()` every period. If the path is only known later (common in ROS nodes, where the path depends on

parameters or on the log directory), create the recorder without one and call `rec.start_autosave(period, path)` once it is known (`test_start_autosave_later`). The autosave thread never raises into the host process: a failed flush prints `[flight_recorder] autosave failed: ...` and the thread tries again next period. `save()` stops the thread, joins it (5 s timeout) and does the final flush. `start_autosave` may be called again after `save()` to keep recording into the same file.

Final flush at exit. Once a recorder has written its file (first flush), an `atexit` hook, which holds only a weak reference, calls `save()` at interpreter exit if the recorder still exists. Rows appended after the last autosave are therefore not lost when a node forgets `save()` or exits normally. A crash or `kill -9` still loses up to one autosave period.

C++ has no autosave thread. Call `rec.flush()` periodically, e.g. from a 1 Hz ROS timer in its own callback group, so that it runs concurrently with the control timer. `~fr::Recorder()` does a final flush when the recorder has a path.

7.3 Caveats

- **HDF5 is not a journaling format.** A crash *during* a flush can leave the file unreadable. Flushes are short (milliseconds for a few seconds of data). If that risk matters, copy the file after each flush, or also record the raw topics with `rosbag2/MCAP`.
- **Flush cost is mostly gzip.** Compressing and writing $30\,000 \times 20$ values plus 600 plans in one go takes about 250 ms (Python) or 160 ms (C++). A periodic flush costs proportionally less.
- **In Python, flushes compete with the control loop for the GIL.** `h5py` holds the GIL while it compresses and writes, so prefer frequent, small flushes to rare, big ones.
- **Autosave keeps up even with a busy-waiting writer.** CPython hands the GIL to a waiting thread every 5 ms (`sys.getswitchinterval()`), so the autosave thread runs even if the control loop never blocks. Measured: a 100 Hz loop that busy-waits between ticks had 217 of 220 rows on disk after 2.2 s (autosave period 0.2 s), and a loop appending as fast as possible had 4.37 M of 5.27 M rows on disk. (An earlier draft of this guide claimed the opposite; that measurement did not reproduce.)

8 Python API

8.1 Recorder

`Recorder(path=None, metadata=None, autosave_period=None, compression_level=4)`

Table 5: Recorder API (`flight_recorder/recorder.py`).

member	what it does
<code>stream(name, columns, capacity=4096) -> ColumnBuffer</code>	declare a stream once, at init; keep the returned buffer. Duplicate names and empty or duplicate columns raise <code>ValueError</code>
<code>rec[name]</code>	the <code>ColumnBuffer</code> of an existing stream
<code>record(group, key, arrays, attrs=None, copy=False)</code>	store rarely changing arrays once under records/<group>/<key>. The arrays are converted with <code>np.asarray(..., float)</code> and kept by reference until the next flush : pass data you won't modify, or <code>copy=True</code> . An array and an attribute with the same name raise <code>ValueError</code>
<code>event(t, kind, detail='')</code>	a timestamped string pair

member	what it does
<code>set_metadata(**values)</code>	add or replace root attributes (written at every flush)
<code>flush(path=None) -> str</code>	append everything new to the file; safe from another thread. Returns the path
<code>save(path=None) -> str</code>	stop autosave, then do a final flush
<code>start_autosave(period, path=None)</code>	start the background flush thread (again after <code>save()</code> if needed)
context manager	with <code>Recorder(path)</code> as <code>rec</code> : ... calls <code>save()</code> on exit if a path is set

Rules worth knowing:

- **One recorder writes one file.** After the first flush, flushing to a different path raises `ValueError('already writing ...; a recorder writes one file')`. Later flushes append only the new rows at their offsets, so another file would miss the earlier ones.
- **HDF5 extensions are kept; anything else gets .h5 appended.** `run.h5`, `run.hdf5`, `run.hdf` and `run.he5` are used as given; `run` becomes `run.h5` and `run.csv` becomes `run.csv.h5` (nothing is silently dropped). C++ uses the same rule. The returned path, and `rec.path` afterwards, are the real one.
- **The first flush replaces an existing file** at that path (mode "w").

Helpers: `record_key(key)` gives the zero-padded key string, and `git_commit(path)` gives the short hash of the git repository containing path, or ''. It is handy as `metadata: metadata={'git_commit': git_commit(__file__)}`.

8.2 ColumnBuffer

Returned by `rec.stream(...):` `append(*values)`, `append_array(row)`, `len(buf)`, `buf.width`, `buf.columns`, `column(name)` (a view), `rows(start=0, stop=None)`, `as_dict()` and `to_dataframe()` (copies). Section 5 explains its internals.

8.3 FlightLog

```
with FlightLog('flight.h5') as log:
    log.metadata                # dict of root attributes (bytes decoded, NumPy
    ↪ scalars as Python values)
    log.streams                 # ['estimator', 'ticks']
    log.columns('ticks')       # names in declaration order
    x = log.column('ticks', 'x') # one column, without reading the others
    df = log['ticks']           # == log.stream('ticks'): pandas DataFrame
    log.record_groups           # ['plans']
    log.record_keys('plans')    # ['000000', '000001', ...] (sorted)
    p = log.record('plans', 12) # {'reference': array, 't_start': 18.0, 'horizon_s':
    ↪ 3.0}
    log.events                  # DataFrame(time, kind, detail)
    print(log.summary())
```

`record(group, key)` returns **one dict** that merges the arrays *and* the attributes, so an attribute with the same name as an array overwrites it in the dict. `FlightLog` opens the file read-only and raises `ValueError` if the format attribute is not "flight_recorder". For the synthetic flight, `summary()` prints:

```

synthetic_flight.h5 (writer: python, created 2026-09-28T17:23:30)
  stream estimator          600 rows x 3 columns: time, bias_x, bias_y
  stream ticks              6000 rows x 9 columns: time, x, y, z, x_ref, y_ref,
↳ z_ref, thrust ...
  records plans              40 entries
  events                     3
  metadata: {'controller': 'twinflight', 'created': '2026-09-28T17:23:30', 'gains':
↳ array([6. , 4.5, 1. ]), 'host': 'example-host', 'rate_hz': 100.0, 'robot':
↳ 'skie_2', 'writer': 'python'}

```

8.4 A complete example

This is, in short, what `make_figures.py` does to produce Figure 7: a 100 Hz “ticks” stream, a second 10 Hz stream, a plan recorded every 1.5 s with the plan number logged per tick, three events, and a flush every 5 s of flight.

```

import numpy as np
from flight_recorder import Recorder, FlightLog

rec = Recorder('synthetic_flight.h5',
               metadata={'robot': 'skie_2', 'rate_hz': 100.0, 'gains': [6.0, 4.5,
↳ 1.0]})
ticks = rec.stream('ticks', ['time', 'x', 'y', 'z', 'x_ref', 'y_ref', 'z_ref',
↳ 'thrust', 'plan_seq'],
                  capacity=6000)
est = rec.stream('estimator', ['time', 'bias_x', 'bias_y'], capacity=600)
seq = -1
for k in range(6000):
    t = 0.01 * k
    ...                                     # state estimate, reference,
    ↳ control
    if k % 150 == 0:                         # a new plan: stored ONCE, in
        ↳ full
        seq += 1
        rec.record('plans', seq, {'reference': plan}, {'t_start': t, 'horizon_s': 3.0})
    ticks.append(t, *pos, *ref, thrust, seq)    # per tick: numbers only (+
    ↳ which plan)
    if k % 10 == 0:
        est.append(t, bx, by)
    if k == 3100:
        rec.event(t, 'mode', 'gust detected')
    if k % 500 == 499:
        rec.flush()                         # or autosave_period=5.0
rec.save()

```

8.5 Analysis recipes

```

import pandas as pd
from flight_recorder import FlightLog

```

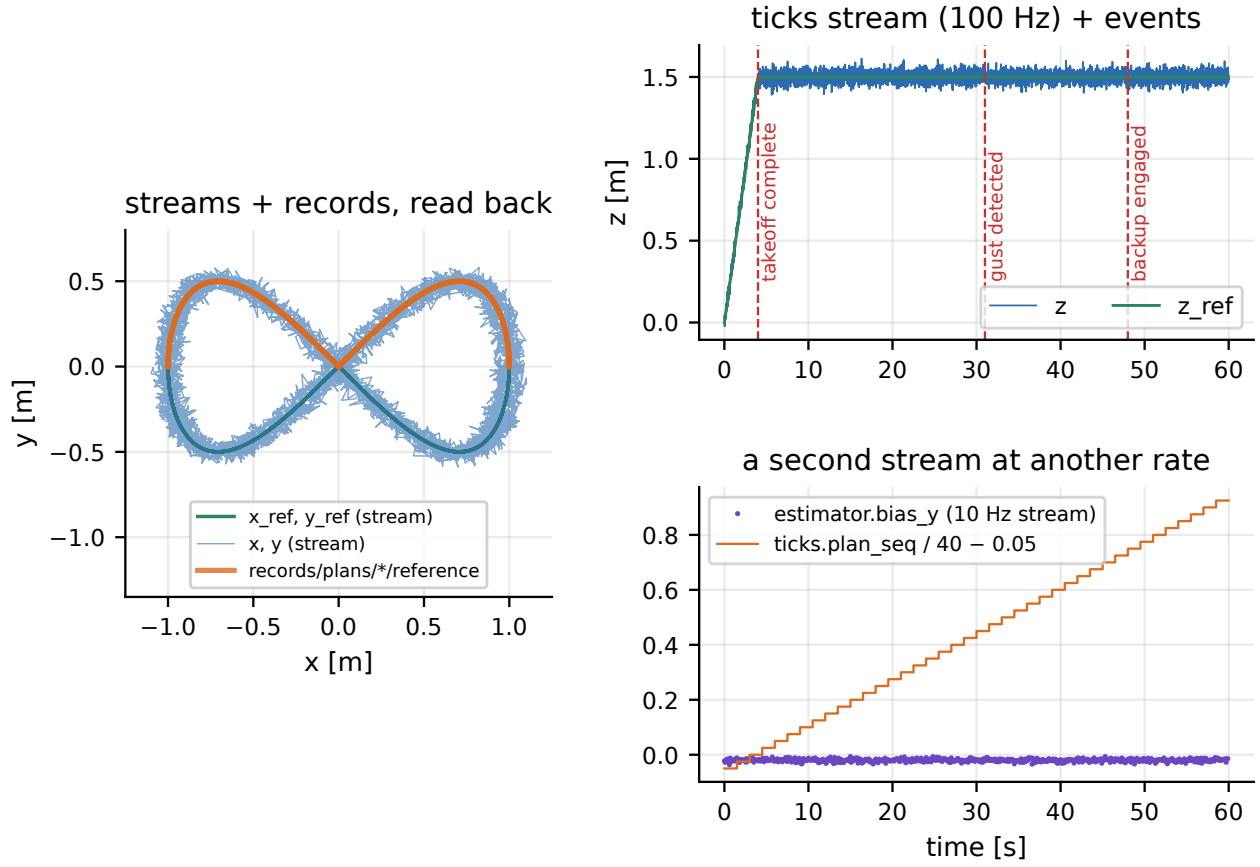


Figure 7: The synthetic flight read back with FlightLog. The tick stream (blue) with its reference (green), every fourth stored plan (orange, from records/plans/*/reference), the events as dashed lines, and a second stream at 10 Hz next to the per-tick plan_seq column.


```

with FlightLog('run.h5') as log:
    ticks, est = log['ticks'], log['estimator']
    merged = pd.merge_asof(ticks, est, on='time') # streams at different
    ↪ rates, aligned
    plan = log.record('plans', int(ticks['plan_seq'].iloc[500])) # the plan active at
    ↪ tick 500
    ev = log.events[log.events['kind'] == 'mode'] # filter events
    ticks.to_parquet('run_ticks.parquet') # hand off to an
    ↪ analysis pipeline

```

9 C++ API

include/flight_recorder/recorder.hpp is header-only and uses the HDF5 C API directly: no HDF5 C++ bindings, no HighFive. Its only dependency is libhdf5-dev. Everything is in namespace fr.

Table 6: The C++ writer (fr namespace).

item	details
fr::Recorder(std::string path = "", const Attributes& metadata = {}) Stream& stream(name, columns, capacity = 4096) stream.append(v...) / append_row(const double*, size_t) ~Recorder()	sets format, format_version, writer="cpp", created, host, then your metadata the reference stays valid for the recorder's lifetime; a duplicate name throws std::invalid_argument hot path; the number of values must equal the number of columns final flush if a path is set (errors are printed to stderr, never thrown)
record(group, std::string key, std::map<std::string, Array>, Attributes = {}) record(group, int64_t key, ...) event(double t, kind, detail = ""), set_metadata(key, Value)	string keys are used as they are integer keys zero-padded to 6 digits under the data mutex
std::string flush(const std::string& path = "") std::string save(const std::string& path = "")	append everything new; may run on another thread; one file per recorder the same as flush (the final one)
fr::Array(const double* data, {dims...}), fr::Array(std::vector<double>, {dims...})	a row-major N-d array; copies the data. An empty shape {} is a scalar. The vector form throws std::invalid_argument if the size does not match the shape
fr::Value	std::variant<double, int64_t, std::string, std::vector<double>>
fr::Attributes	std::vector<std::pair<std::string, Value>>

Errors. Every failed HDF5 call throws std::runtime_error("flight_recorder: HDF5 call failed: <function>"). While flush() runs, HDF5's automatic error printing is switched off (so problems arrive only

as exceptions), and the previous handler is restored afterwards (RAII detail::QuietHdf5Errors). Other HDF5 users in the process keep their error reporting. A failed flush puts the unwritten records and events back, so a later flush retries them.

Save on destruction. ~fr::Recorder() does a final flush if the recorder has a path, and prints (never throws) any error. Still call flush() periodically for crash tolerance.

Using it from CMake / colcon:

```
find_package(flight_recorder REQUIRED)
target_link_libraries(my_node flight_recorder::flight_recorder) # include paths +
↳ HDF5 C link line
```

and <depend>flight_recorder</depend> in package.xml. The package also builds two programs: cpp_example <out.h5> [rows] [--concurrent-flush] (the fixture of the C++ round-trip test) and cpp_bench <out.h5> [ticks].

In a ROS 2 C++ node (from docs/MIGRATION.md):

```
#include <flight_recorder/recorder.hpp>
fr::Recorder rec_{path, {{"node", std::string("offboard")}}};
fr::Stream& ticks_ = rec_.stream("ticks", {"time", "x", "y", "z", "yaw"}, 60000);
// control timer: ticks_.append(t, x, y, z, yaw);
// 1 Hz timer in its own callback group: rec_.flush();
// shutdown: rec_.save();
```

10 ROS 2 integration

examples/ros2_node_example.py shows the pattern for any control node:

1. **Declare the streams once in __init__**, with fixed float64 columns and capacity $\approx$ the flight length.
2. **One append per tick** in the control callback, numbers only.
3. **record large arrays only when they change**, and put their sequence number in the tick stream.
4. **save() at shutdown**, in main's finally. autosave_period keeps a crash from losing more than a few seconds.

```
class ExampleController(Node):
    def __init__(self):
        super().__init__('flight_recorder_example')
        self.rec = Recorder('example_flight.h5', autosave_period=5.0,
                           metadata={'node': self.get_name(), 'git_commit':
↳ git_commit(__file__),
                                   'rate_hz': 100.0})
        self.ticks = self.rec.stream('ticks', ['time', 'x', 'y', 'z', 'yaw',
↳ 'u_thrust', 'comp_time'],
                                   capacity=60_000)
        self.create_timer(0.01, self.control)

    def control(self):
        ...
        self.ticks.append(t, x, y, z, yaw, u, time.perf_counter() - c0)
```

```
def main():
    rclpy.init(); node = ExampleController()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        print(node.rec.save())
        node.destroy_node()
```

Practical points:

- **Where to write.** Use one file per node and per run. HDF5 file locking refuses a second writer on the same file. If the path comes from a parameter, use `start_autosave(period, path)` once you know it.
- **Metadata is free.** Put the parameters, gains, git commit and controller mode in `metadata=`, so the file describes itself.
- **Pair it with rosbag2.** `flight_recorder` records what a node computes, from inside it. `rosbag2/MCAP` records what flows on topics between nodes, and can replay it. They are complementary: see `docs/COMPARISON.md`.

11 Command line and CSV export

```
python3 -m flight_recorder info flight.h5          # summary(): streams,
↳ records, events, metadata
python3 -m flight_recorder csv flight.h5 ticks -o ticks.csv  # plain CSV of one
↳ stream
python3 -m flight_recorder ros2logger-csv flight.h5 ticks -o legacy.csv
ros2 run flight_recorder flight_recorder info flight.h5    # the same, through
↳ the installed script
```

Without `-o`, the output is `<file>_<stream>.csv` next to the input. The functions behind the CLI are in `flight_recorder.csv_export`:

- `stream_to_csv(log, stream, path): DataFrame.to_csv(index=False, na_rep='nan');`
- `stream_to_ros2logger_csv(log, stream, path, required=REQUIRED):` one stream in ROS2Logger's layout;
- `write_ros2logger_csv(columns, path, required=REQUIRED) -> (rows, cols):` the general writer. It takes `(header, values, order, tie_key)` tuples and orders the columns with the required ones first (`REQUIRED = ('time', 'x', 'y', 'z', 'yaw')`), then by `(order, tie_key, header)`. Shorter columns are padded with NaN, and values are written by Python's `csv` module (floats as `repr`, NaN as `nan`), exactly like ROS2Logger. A missing required column raises `ValueError('[flight_recorder] Missing required logs: [...])'`.

12 ROS2Logger compatibility and migration

`flight_recorder.compat` is a drop-in replacement for ROS2Logger's API. Change one import:

```
# from ros2_logger import Logger, LogType, VectorLogType, install_shutdown_logging
```

```
from flight_recorder.compat import Logger, LogType, VectorLogType,
↳ install_shutdown_logging
```

Table 7: The compat layer (flight_recorder/compat.py).

class / function	behaviour
LogType(name, order, capacity=4096)	scalar log: append(value), extend(values), .data (a copy), len()
VectorLogType(name, order, subnames, capacity=4096)	one column per sub-name: append(*values) (checks the count), append_dict(**kw); CSV headers <name>_<subname>
Logger(filename, base_dir, required=REQUIRED, write_h5=True)	ROS2Logger's path convention .../src/data_analysis/log_files/<pkg>/<filename> (build/install mapped to src)
Logger.log(source)	finds every LogType/VectorLogType attribute of source (via vars()), writes the byte-identical CSV, and, if write_h5, an .h5 next to it with one stream per log (sharing the buffers, not copying them)
install_shutdown_logging(logger, source, *, also_shutdown=None, include_atexit=True)	logs once on SIGINT/SIGTERM (and at exit), then destroy_node() and rclpy.shutdown() or your also_shutdown

The values live in ColumnBuffers, so the compat logs add nothing to the GC's work (test_gc.py). The differences from ROS2Logger: values must be numeric, and ROS2Logger's analysis notebooks are not copied into the log directory. test_compat.py compares the CSV byte for byte with ROS2Logger's when ros2_logger is importable.

The real gains come from the native API, as docs/MIGRATION.md explains step by step:

Table 8: Migration, conceptually.

ROS2Logger habit	native flight_recorder
one LogType per variable, all in one wide table	one stream per rate or producer
re-logging slices of an array every tick	record the array once when it changes; log its id per tick
strings in logs	events
parameters hard-coded in notebooks	metadata in the file (git_commit())
write at shutdown only	autosave_period= (Python) or a periodic flush() (C++)

Notebooks that expect the old CSV can be fed from the new files with flight_recorder ros2logger-csv run.h5 ticks -o run.csv.

13 Benchmarks

benchmarks/bench_logging.py flies the same synthetic 5-minute, 100 Hz flight through eight logging methods: 30 000 ticks × 20 float64 values, plus a new 100 × 10 plan every 50 ticks, every plan different so

compression gets no free wins. Each method runs in **its own fresh Python process**, twice: once for timing and once for memory, because allocation tracing (`tracemalloc`) slows every allocation. The noop method measures the harness itself. The numbers below are from `benchmarks/results_python.md` (Python 3.12.3, NumPy 1.26, h5py 3.16, pandas 3.0, 14-core desktop in performance mode).

benchmarks/results_python.md (5-min 100 Hz flight, 20 values/tick, 100×10 plan every 50 ticks)

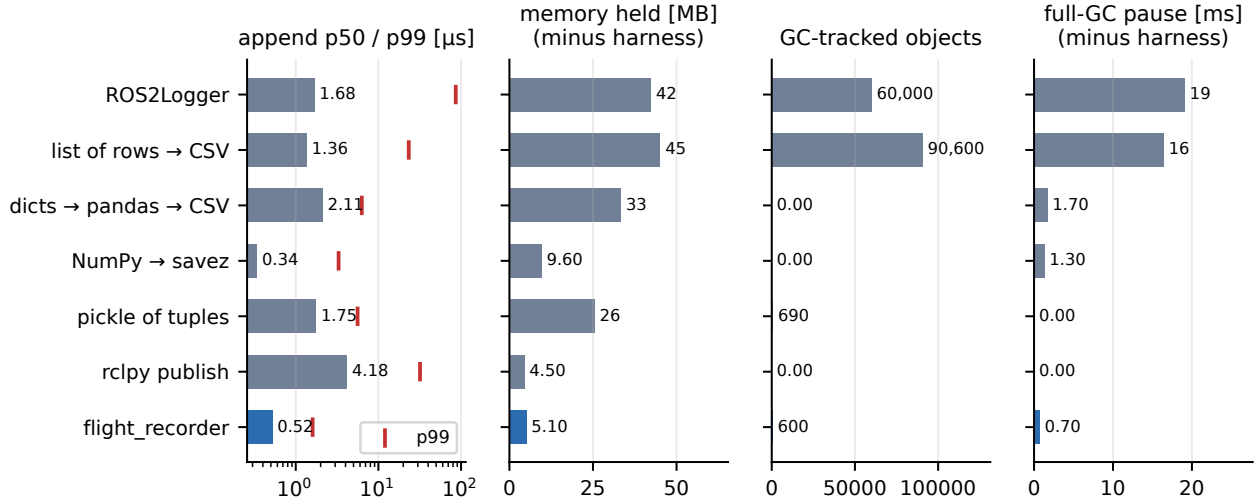


Figure 8: The recorded benchmark results. Memory and full-GC pause are shown after subtracting the harness (noop: 5.1 MB, 10.0 ms). The red ticks on the first panel mark p99.

Table 9: Benchmark summary. Memory is after subtracting the harness; rclpy publishing is saved by another process (rosbag2).

method	append p50 / p99 [μs]	memory [MB]	GC-tracked	save [ms]	file [MB]	read [ms]
ROS2Logger (LogType lists → CSV)	1.68 / 86.3	42.4	60 000	1281	26.0	151
list of rows → CSV	1.36 / 23.3	45.1	90 600	305	11.8	67
list of dicts → pandas → CSV	2.11 / 6.3	33.4	0	402	11.8	67
NumPy preallocated → savez_compressed	0.34 / 3.3	9.6	0	243	9.2	23
pickle of tuples	1.75 / 5.6	25.5	690	15	10.3	51
rclpy publish per tick (→ rosbag2)	4.18 / 31.9	4.5	0	—	—	—
flight_recorder (Python)	0.52 / 1.6	5.1	600	246	11.3	17
flight_recorder (C++)	0.015 / 2.3	4.8 (buffer)	—	158	11.3	—

How to read it:

- **Append.** `flight_recorder` is as cheap as a raw preallocated NumPy row write, and its p99 is the lowest of all (1.6 μs). The long tails of the list-based methods (23–86 μs) are allocation and GC work landing inside the loop.

- **Memory and GC.** ROS2Logger holds 42 MB for 4.8 MB of numbers, and lists of rows add 90 600 tracked objects. flight_recorder's 600 tracked objects are one small entry per stored plan. The recorder keeps every record (with its arrays) for its whole lifetime (section 15).
- **Full-GC pause.** The column is one full collection at the end of a small benchmark process. What matters is the difference from the harness: +19 ms for ROS2Logger, +16 ms for lists, and nothing for flight_recorder. In a real node the whole heap is scanned, so the absolute pauses are far larger (≈ 300 ms in the RTA node).
- **File size.** Random test data does not compress, so the per-tick sizes are close to each other here. On a real flight the per-tick table is 0.36 MB in flight_recorder against 1.28 MB as CSV. The whole ROS2Logger-layout CSV of that flight was 5.42 MB, while flight_recorder stored it *plus* every full plan in 1.61 MB.
- **Read back.** 17 ms, because loading is decompression plus float64 arrays: no text parsing.

To reproduce: `python3 benchmarks/bench_logging.py --out results.md` (methods whose dependency is missing, e.g. `ros2_logger`, are reported as skipped). For C++: `cpp_bench out.h5` prints the append percentiles and the save time as JSON.

14 Testing

Table 10: The 31 test cases.

test	checks
<code>test_recorder.py::test_round_trip</code>	bit-exact float64 round trip (a tiny capacity forces growth), records, events, metadata types
<code>...::test_incremental_flush_equals_single_array</code>	many flushes == one save (streams and events)
<code>...::test_rerecording_a_key_replaces_it</code>	a record key written again after a flush is replaced
<code>...::test_concurrent_flush_sees_only_completed</code>	50000 appends against a tight-loop flusher thread; every value checked
<code>...::test_autosave_writes_periodically</code>	autosave writes; the file is readable while the recorder runs
<code>...::test_crash_after_flush_keeps_flushed_data</code>	<code>data_exit(1)</code> after a flush leaves exactly the flushed rows
<code>...::test_rejects_wrong_row_width</code>	wrong row width raises
<code>...::test_one_recorder_one_file</code>	flushing to a second path raises
<code>...::test_start_autosave_later</code>	autosave started after the streams, once the path is known
<code>...::test_append_rejects_wrong_width_instead_of_broadcast</code>	single value is not broadcast over all columns (append, <code>append_array</code>)
<code>...::test_hdf5_extension_kept_otherwise_appended</code>	extension policy (5 cases)
<code>...::test_autosave_can_restart_after_save</code>	<code>start_autosave</code> after <code>save()</code> really flushes
<code>...::test_records_and_events_released_after_flush</code>	flushed records and events leave memory; later ones still write
<code>...::test_failed_flush_keeps_pending_records</code>	defailing flush loses nothing
<code>...::test_record_rejects_array_attribute_names</code>	list of names raise
<code>...::test_reader_rejects_newer_format_versions</code>	clear error for newer files
<code>...::test_set_metadata_while_autosaving</code>	metadata updated concurrently with autosave flushes
<code>...::test_final_flush_at_interpreter_exit</code>	rows appended after the last flush reach the file at exit, with no <code>save()</code>

test	checks
test_cpp_edge_cases.py	C++: extension policy, HDF5 error handler restored, destructor flush, scalar arrays, size/shape check, name clash, records released
test_gc.py (3 tests)	stream and compat appends add no tracked objects; list logging does (contrast)
test_compat.py (3 tests)	CSV byte-identical to ROS2Logger (skipped without <code>ros2_logger</code>); the <code>.h5</code> side file; missing required logs raise
test_cpp_roundtrip.py (2 cases)	the C++ writer's file reads back exactly in Python, with and without a concurrent flush thread

With `colcon`, `colcon test --packages-select flight_recorder && colcon test-result --verbose`. The CMake test registration passes `CPP_EXAMPLE`, `CPP_EDGE_CASES` and `PYTHONPATH` for you. Without `colcon`:

```
g++ -O2 -std=c++17 -Iinclude -I/usr/include/hdf5/serial examples/cpp_example.cpp \
-L/usr/lib/x86_64-linux-gnu/hdf5/serial -lhdf5 -pthread -o /tmp/cpp_example
CPP_EXAMPLE=/tmp/cpp_example PYTHONPATH=. python3 -m pytest -q test
```

When this guide was built (v0.1.2): **30 passed, 1 skipped** (the byte-identical ROS2Logger comparison, because `ros2_logger` was not installed). The C++ programs were also run under AddressSanitizer + UBSan and, with a concurrent flush thread, ThreadSanitizer: no findings.

15 Limitations, gotchas and FAQ

15.1 By design

- **Streams are float64 only.** Integers and booleans are exact up to 2^{53} . Strings go in events or metadata, arrays in records.
- **Columns are fixed per stream.** To log something new, declare another stream.
- **Not a middleware recorder.** Use `rosbag2` for topics.
- **One writer process per file.** Different nodes should use different files.
- **A crash during a flush can corrupt the file** (section 7).

15.2 Behaviours to know about

- `record()` keeps its arrays **by reference** (unless `copy=True`). Modifying a plan array in place before the next flush changes what gets written.
- Records and events are held in memory only **until they are flushed**, then released (both writers). Between flushes they accumulate, so don't use `record()` as a per-tick stream.
- Re-recording a key deletes and rewrites its HDF5 group. HDF5 does not reclaim the freed space, so re-recording large arrays many times makes the file grow; `h5repack` compacts it.
- The first flush truncates an existing file at the same path (a recorder starts a new flight).
- Array and attribute names of one record share a namespace: a clash raises at `record()`. For files written by 0.1.1 or earlier, `FlightLog.record` keeps the array, warns, and `record_attrs()` returns the attributes.

Fixed in 0.1.2: a single value passed to a multi-column append was silently broadcast over all columns; `start_autosave` after `save()` started a thread that exited at once; flushed records and events stayed in memory; concurrent `set_metadata` and `flush` were unsynchronised; the reader ignored `format_version`; Python replaced any extension with `.h5`; C++ disabled HDF5 error printing process-wide, had no destructor flush, could not write scalar arrays, and did not check Array size against shape (an out-of-bounds read).

15.3 FAQ

How large should capacity be? About the number of rows in the flight: $\text{rate} \times \text{duration}$. It is only a performance hint in Python (doubling beyond it) and in C++ (one block allocation every 4096 rows beyond it).

Can I read the file during the flight? Yes, between flushes. The file is closed and consistent then, and `FlightLog` opens it read-only.

How do I log a boolean or a mode? As a float column (1.0/0.0, or a mode index). For a textual transition, also log an event.

Different rates? Use one stream per rate and join them with `pd.merge_asof(..., on='time')`.

I need Parquet / MATLAB / CSV. `log['ticks'].to_parquet(...)`; MATLAB and Julia read the HDF5 directly; the CLI writes CSV.

How much data is lost in a crash? At most one autosave period (Python) or one flush interval (C++). The in-memory buffer since the last flush is lost; everything before it is on disk.

16 Reproducing this guide

```
cd docs/guide
python3 make_figures.py          # needs numpy, h5py, pandas, matplotlib; ~1 min
quarto render flight_recorder_guide.qmd
```

`make_figures.py` writes every figure as PDF with TrueType (Type 42) fonts, never Type 3, plus the Markdown snippets included above: the real file tree, the `summary()` output and the measured hot-path table. The hot-path table and Figure 5 are re-measured on every build, so they vary with the machine; the benchmark figure and table come from `benchmarks/results_python.md`.